

RACER: Residual-Adaptive Closed-Loop Estimation for Sampling-Based Planning in Wheeled-Quadruped Racing

Anonymous Authors

Abstract—We present RACER, a hierarchical control framework for wheel-based quadruped racing that combines an MPPI planner with a learned residual dynamics model and a low-level RL velocity tracker. The planner augments a nominal unicycle kinematic model with a neural residual term to capture the closed-loop tracking behavior of the RL policy. To train this residual model under limited real-world data, we propose Low-Rank Residual Adaptation (LoRRA), a two-stage approach that pre-trains on large-scale simulation data for broad coverage and then fine-tunes on a small real-world dataset with a low-rank constraint. In simulation, we empirically validate our engineering choices by showing (A) Residual dynamics improve the overall performance of our pipeline by capturing the tracking error of RL velocity tracker at high-speed cornering. (B) Residual dynamics trained with both source-domain and target-domain data gives racing performance significantly better than the residual dynamics trained with only target-domain data. (C) Low-rank constraint at target-domain adaptation gives higher success rates and higher performance than full-tune and from-scratch when domain gap in ground coefficient or joint gain increases.

I. INTRODUCTION

Racing a wheeled quadruped is challenging. First, wheeled quadruped inherits the high-dimensional dynamics of a quadruped with the nonlinear tire dynamics of a racing car [1, 2]. This complexity challenges many classical model-predictive control (MPC) approaches: optimizing long-horizon trajectories over such a high-dimensional and non-linear system is challenging [3, 4]. At the same time, long-horizon is a must in the context of racing [5]. This long-horizon requirement makes naive end-to-end reinforcement learning (RL) approaches equally impractical [6]. A long planning horizon creates an extremely sparse reward landscape that destabilizes RL training, and even if clever curriculum or reward-shaping techniques succeed in training an agent, the resulting policy rarely generalizes beyond its training distribution.

To handle the system complexity and long-horizon nature of the racing task, we adapt hierarchical control. An RL policy can handle the nonlinear, high-dimensional dynamics of the wheeled quadruped over a short horizon. It is trained to track velocity commands. It abstracts the full joint-space dynamics into a compact, low-dimensional interface of linear and angular velocity [2, 7, 8]. A sampling-based planner [9, 10], can then operate over this interface to plan trajectories across a long horizon.

While this hierarchical architecture is conceptually sound, a naive implementation will fail at high-velocity (Sec. IV-A). We find that if the planner uses simple reduced-order models to propose trajectories [1, 4], the resulting commands will systematically diverge from the robot’s true behavior, because velocity tracking is always imperfect: hardware latency, ac-

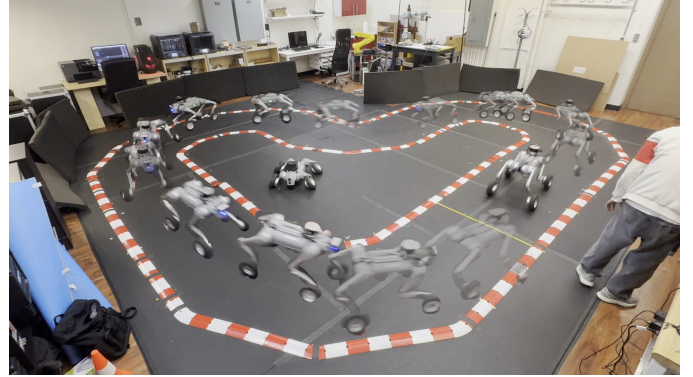


Fig. 1: A Go2W Wheeled Quadruped racing along L-shape track using RACER framework. The low-level controller is a velocity RL-tracking policy; the high level planner is a DIAL-MPC controller equipped with residual dynamics trained with LoRRA.

tuator saturation, and tire slip all introduce tracking errors that a kinematic model cannot predict. To bridge this gap, we augment the reduced-order model (namely the unicycle model) with a learned residual dynamics network that compensates for the difference between the analytic kinematic prediction and the actual closed-loop next state of the RL tracker [11, 12]. Training such a residual model, however, is highly nontrivial since it requires both broad data coverage across the state-action space and high data quality that reflects real-world dynamics. Simulation can provide coverage but suffers from the sim-to-real gap [13, 14]; real-world data provides fidelity but is scarce and expensive to collect on a fragile racing platform.

To this end, we propose Low-Rank Residual Adaptation (LoRRA), a two-stage training pipeline that first pre-trains the residual network on a large simulated dataset for coverage (10,000,000 state transitions). By coverage we mean that these transitions span the range of velocities, yaw rates, and commands the planner queries during high-speed rollouts, so the residual is accurate across the states DIAL-MPC actually visits rather than only near a handful of collected trajectories. Then, we fine-tune it on a small real-world dataset (about 400 state transitions) under a low-rank parameter constraint that prevents catastrophic forgetting of the simulation prior [15, 16]. Our contribution are as follows:

- We propose Residual-Adaptive Closed-Loop Estimation for Sampling-Based Planning in Wheeled-Quadruped Racing (RACER), a hierarchical control stack for wheeled-quadruped racing.

- We propose LoRRA, a simple yet effective training pipeline for residual dynamics that overcomes the sim-to-real gap.
- We conduct exhaustive empirical experiments to validate the functionalities of residual dynamics and LoRRA.

II. RELATED WORK

a) Wheeled-legged locomotion and racing.: Bjelonic et al. [1] introduce online trajectory optimization for hybrid walking-driving locomotion on ANYmal-on-wheels, and extend it to whole-body MPC with single-rigid-body dynamics. [4]. Lee et al. [2] take a complementary learning-heavy direction, integrating RL locomotion, learned high-level navigation, and large-scale path planning for kilometer-scale urban autonomy. These works mainly focus on hybrid locomotion and urban traveling while high-speed racing with wheeled quadruped is still an engineering setting largely underexplored. The only work to directly address wheeled-quadruped racing [17] approaches it from a vehicle-dynamics standpoint. In contrast, we focus on functionalities of residual dynamics. [17] and our approach serve as complementary roles.

b) Learned and residual dynamics in sampling-based MPC.: Sampling-based MPC, and MPPI in particular [9], is a natural consumer of learned dynamics because it only requires forward simulation. Roth et al. [11] learn a perceptive forward dynamics model on ANYmal and roll it out inside MPPI for safe navigation, reporting a 41% improvement in position prediction; Kim et al. [12] pair a learned quadruped forward model with an informed trajectory sampler for safe local planning. On the ground-vehicle side, Kato et al. [18] fuse a physical model with a neural residual and use MPPI to optimize PID gains for a mini-forklift, and Song et al. [19] learn a continuous-time residual via input-concomitant Neural ODEs. Pezzato et al. [20] sidestep model learning by treating IsaacGym itself as the rollout simulator, while Xue et al. [10] push sampling MPC to torque-level quadruped control using full-order dynamics and a diffusion-style annealing schedule that we adopt for our planner. Relative to this body of work, our residual is distinctive in two ways. First, it is parameterized at the planner’s *command interface* (rather than at the full robot state); second, its training pipeline is dual-phased, consuming both simulation data and hardware data.

c) Sim-to-real adaptation of dynamics models.: Heiden et al. [21] augment a differentiable rigid-body simulator with neural networks that absorb unmodeled contact effects; Cheng et al. [22] tune controller parameters through auto-differentiation, and Krishna et al. [23] co-tune controller and simulator parameters from a few deployment rollouts. He et al. [24] learn a *delta-action* residual on a humanoid to realign IsaacGym with the deployed robot; Levy et al. [25] learn semi-structured dynamics from three minutes of real Go1 data to bootstrap model-based RL from scratch. These works exclusively focus on the sim-to-real of policy, and to the best of our knowledge, we are the first work that studies the sim-to-real transfer of residual dynamics. [26] is the closest work. [26] freezes a symbolic-regression base learned in low-fidelity

simulation and fits a fresh neural residual per target domain, at the level of open-loop plant dynamics queried directly by MPC. In contrast, RACER (i) parameterizes its residual at the planner’s command interface to model the closed-loop tracking error of a learned RL velocity tracker, and (ii) transfers the pretrained residual itself across the domain gap under a low-rank constraint, rather than re-fitting a residual from scratch.

d) Parameter-efficient fine-tuning and adaptation regimes.: Low-Rank Adaptation [15] freezes pre-trained weights and trains low-rank updates that can be merged with no runtime cost; in robot learning, Daneshmand et al. [16] combine LoRA with a recovery-policy safety mechanism to fine-tune a Unitree Go2 locomotion *policy*, reporting 46.5% lower fine-tuning time and rank-1 adapters sufficing to recover pre-trained real-world performance. To the best of our knowledge, we are the first work that investigates the impacts of low rank constraint on the training of residual dynamics rather than policy adaptation.

III. HIERARCHY OVERVIEW: RACER

RACER employs a two-level hierarchical control architecture: (1) a DIAL-MPC planner that computes optimal velocity commands (More details about the planner’s cost function in Appendix Sec. A), and (2) an RL velocity tracker that executes these commands (Sec. III-A).

Our primary contribution lies in the planner layer. To roll out trajectories, the planner uses a discrete-time planar kinematic model augmented with residual dynamics:

$$s_{t+1} = f_{\text{uni}}(s_t, a_t) + g_{\phi}(s_t, a_t), \quad (1)$$

where the state $s_t = [x_t, y_t, \psi_t, v_{x,t}, \omega_{z,t}]^T$ comprises the quadruped’s base-link world-frame position, yaw, body-frame longitudinal velocity, and yaw rate. The action $a_t = [v_{x,t}^{\text{des}}, \omega_{z,t}^{\text{des}}]^T$ represents the desired velocity command passed to the low-level RL tracker. The term f_{uni} denotes a nominal kinematic unicycle model, while g_{ϕ} is the learned residual dynamics. Equation 1 models the closed-loop behavior of the RL tracker, predicting the resulting next state s_{t+1} given the current state s_t and the commanded action a_t . In this work, g_{ϕ} is implemented as a 5-layer MLP sharing the same input and output space as f_{uni} .

A. Low-Level RL Velocity Tracker

The low level RL velocity tracker performs joint-space locomotion control to track the velocity command given by the planner. Specifically, it is a feed-forward neural policy mapping a proprioceptive observation (joint angle, joint velocity, base-link pose) and the planner velocity command to a 16-D joint-actuator output: 12 leg-joint position targets and 4 wheel-velocity targets. Its training follows the standard PPO paradigm [27]. Remarkably, the reward function of RL training is specifically designed to encourage RL tracker to behave closely to f_{uni} . Thus, the residual dynamics can be interpreted as an estimation of tracking error of this RL policy, bridging the gap between ideal kinematic commands and the inevitable imperfections of real-world execution caused by hardware limitations.

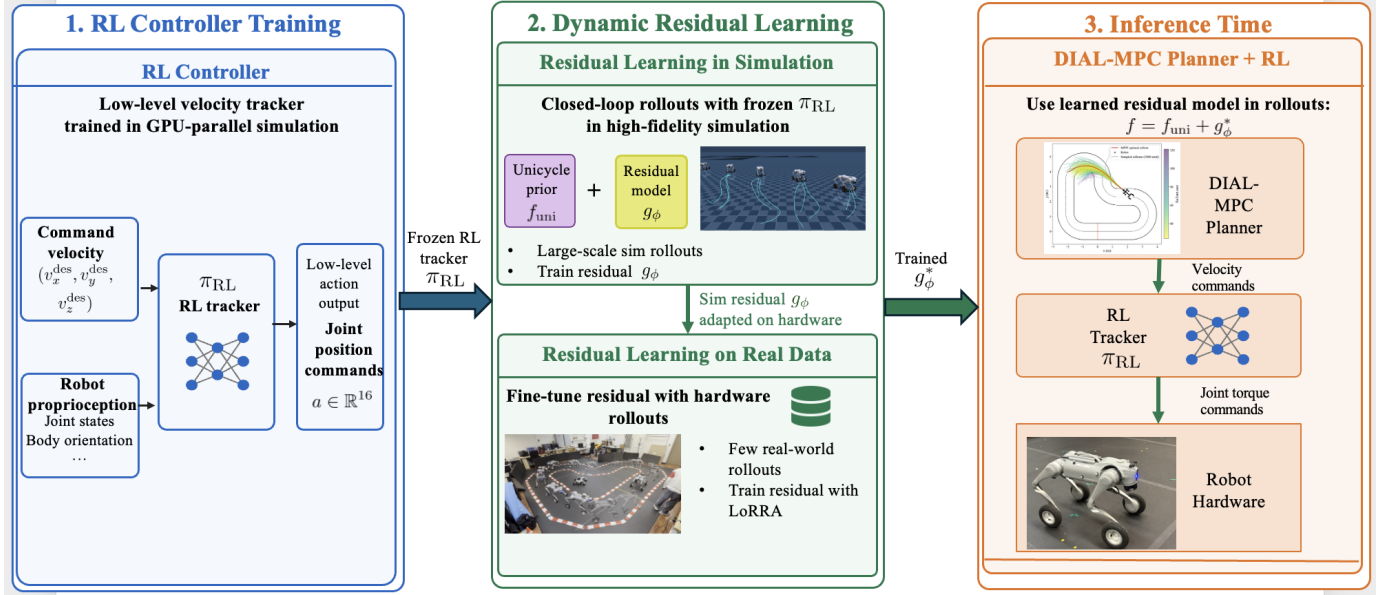


Fig. 2: **System overview.** An DIAL-MPC planner samples velocity commands at 10Hz and rolls them out under $f = f_{\text{uni}} + g_{\phi}^*$. An RL tracker realises the chosen command at 50Hz, producing 16-D joint targets for the Go2-W’s 12 leg joints and 4 wheel motors. Hardware adaptation refines *only* g_{ϕ}^* to compensate domain shift, leaving the planner and tracker untouched.

B. Training Residual Dynamics

g_{ϕ} is trained with two stages: a simulated pretraining stage on a source-domain dataset $\mathcal{D}_s$ that provides data coverage, followed by an on-robot refinement stage on a small target-domain dataset $\mathcal{D}_t$, where $|\mathcal{D}_s| \gg |\mathcal{D}_t|$ with a Low Rank constraint. In sim-to-real transfer literature [28] [13], the *source domain* refers to the accessible training environment (here, simulation), the *target domain* is the final deployment environment (here, real-world quadruped hardware deployment), and the *domain gap* is the discrepancy in transition dynamics between the two.

Both stages minimize the residual MSE loss

$$\mathcal{L}(\phi; \mathcal{D}) = \mathbb{E}_{(s,a,s') \sim \mathcal{D}} \|g_{\phi}(s,a) - (s' - f_{\text{uni}}(s,a))\|_2^2, \quad (2)$$

For the rest of this paper, we will refer to our residual dynamics training recipe as **LoRRA**, Low-rank Residual Adaptation.

Standard class-balancing techniques for single-stage joint training fail due to the severe dataset imbalance ($|\mathcal{D}_s| \gg |\mathcal{D}_t|$) [29, 30]. Naively re-weighting the target loss induces conflicting gradients: when simulation and hardware data present conflicting physical outcomes for similar states, the network compromises by learning an inaccurate average dynamic [31, 32].

a) Low-Rank Adaptation: Our use of low-rank constraints during target-domain adaptation is based on current large language model (LLM) methods [33–35]. We constraint the degrees being updated during $\mathcal{D}_t$ adaptation to be less than rank- r . The critical challenge in this dual-phase training is catastrophic forgetting. If we perform full-parameter fine-tuning on the small target-domain dataset $\mathcal{D}_t$, the network will catastrophically forget about the posterior learned from the

simulation dataset $\mathcal{D}_s$. Previous works have widely validated that low-rank constraint during post-training can keep fine-tuned LLMs closer to the pre-training prior [33–35]. In the context of residual dynamics, the low-rank constraint freezes the pre-trained base weights and confines the target-domain updates to a restricted rank- r subspace. This ensures that the network can absorb hardware-specific corrections without forgetting priors learned from $\mathcal{D}_s$.

More details about low-rank constraint is in Appendix. B.

IV. EXPERIMENTS

The experiments validate our two main methods: residual dynamics and LoRRA. More specifically, we verify the following claims:

- The proposed residual dynamics model improves racing performance of a wheeled-legged robot (Sec. IV-A).
- The proposed dual-phase training, LoRRA, improves racing performance of a wheeled-legged robot under large domain gaps (Sec. IV-B).

Following the autonomous racing convention [5], we report three metrics to quantify racing performance. (1) velocity averaged across laps. A lap’s velocity will be punished to 0 if undesirable contacts happen during the lap (2) maximum deviation from the reference trajectory (Appendix Sec. A). (3) the number of collision-free laps (free of collisions and track-boundary violations) completed within a fixed time.

A. Ablation Study of Residual Dynamics

In this section, we define naive unicycle to be the variant of RACER with residual dynamics ablated. The naive unicycle model is equivalent to RACER except that the naive unicycle’s planner does not use residual dynamics.

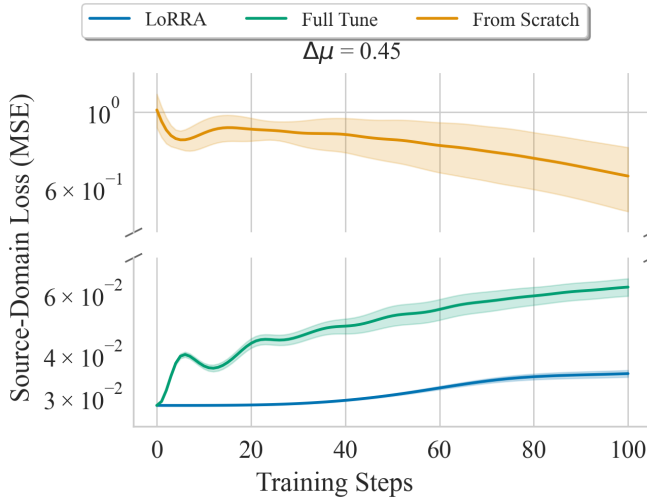


Fig. 3: The Source-Domain loss of residual dynamics $\mathcal{L}(\phi, \mathcal{D}_s)$ through the training with target-domain data $\mathcal{D}_t$. See Sec. IV-B for definition of each training recipe: LoRRA, Full Tune, and From Scratch. Source and target environments are defined to be two simulation dynamics with different tire frictions, $|\mu| = 0.4$. See Sec. IV-B1 for more details about domain definition. **LoRRA(dual-phase training with low-rank constraint) helps preserve the most source-domain posterior with lowest source-domain loss after target-domain adaptation.**

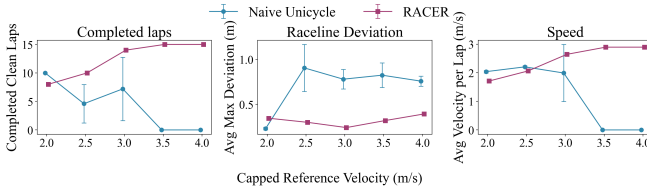


Fig. 4: Simulation performance of Naive Unicycle (Blue) and RACER (Red) v.s. Capped velocity. **RACER has stable performances independent of the velocity cap while the performance of naive unicycle is downgraded as the cap becomes higher.** Horizontal axis: cap velocity. Vertical Axis: the performance metrics. See Sec. IV-A for the definition of naive unicycle. See Sec. IV-A1 for interpretations.

Insight IV.1

Residual dynamics improve the overall performance of our pipeline by capturing the tracking error of RL velocity tracker at high-speed cornering.

1) *Simulation Results:* In simulation, we enforce a range of velocity caps of the format $v_x \leq c$, where c is a constant threshold to the planners. The performance metrics of naive unicycle and RACER versus the magnitude of the velocity caps are reported in Fig. 4. This gives us three observations: i) RACER has low-variance performances regardless of the velocity caps; ii) if we enforce a sufficiently conservative velocity cap to naive unicycle, it can finish clean laps; iii) and as the velocity cap becomes higher, the performance of naive unicycle can be shown downgraded with worse tracking accuracy, higher-chances of failing laps, and lower average velocity.

We propose the following explanation to the observations:

- The RL velocity tracker can accurately track the unicycle

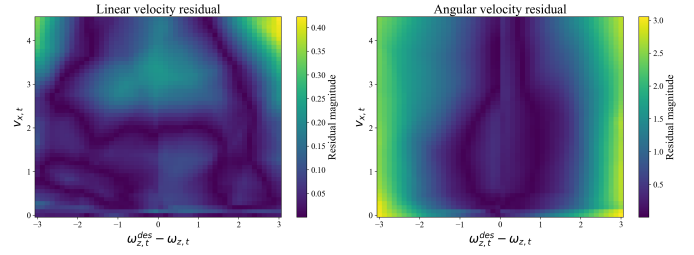


Fig. 5: Residual Magnitude under different linear velocities(horizontal axis) and yaw rate changes(vertical axis). **Residual magnitude positively correlates with linear velocities and yaw rate changes.** Left: Residual of linear velocity; Right: residual of angular velocity. The brighter the area, the larger the residual.

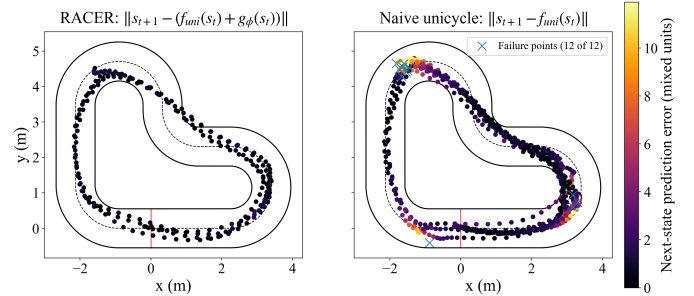


Fig. 6: Next-State Prediction Errors of RACER(left) and Naive Unicycle(right). **Naive Unicycle's prediction errors explode while cornering while RACER keeps the next-state prediction errors consistently low.** Heat Bar: next-state prediction errors. The brighter the points, the larger the prediction errors. Left: $\|s_{t+1} - (f_{\text{uni}}(s_t) + g_{\phi}(s_t))\|$. Right: $\|s_{t+1} - f_{\text{uni}}(s_t)\|$. The trajectories of left are collected using RACER; The trajectories of right are collected using naive unicycle. A cross dot represents a lap failure.

nominal model when the velocity is low.

- The tracking error of the RL velocity tracker explodes at high-velocity cornering. The residual dynamics informs RACER about such errors while naive unicycle is blind to the errors.

We support our explanation with Fig. 6 and Fig. 5. Fig. 6 shows that RACER has consistently-low next-state predictions while Naive Unicycle's predictions errors explode at high-velocity cornering. Per fig. 5, we present the following observations:

- Tracking error generally positively correlates with increases in linear velocity and yaw rate.
- The relationship between the RL agent's tracking error, linear velocity, and yaw rate change is highly non-linear.

All observations above validate our claim about the failure of naive unicycle and outlines the role of residual dynamics g_{ϕ} : capture the highly-nonlinear tracking error of RL velocity tracker, especially at high-speed cornering.

2) *Hardware Results:* On the physical robot we repeat the velocity sweep of Sec. IV-A1, comparing RACER against the naive unicycle at velocity caps $c \in \{1.5, 2.0, 3.2\}$ m/s with three runs each. Table I reports the same three metrics as Fig. 4 and reproduces the simulation pattern. At the conservative 1.5 m/s cap the naive unicycle is even able to finish laps at a slightly higher average velocity. As the cap increases

TABLE I: **Hardware counterpart of Fig. 4.** Racing metrics (mean $\pm$ std over three runs) on the physical robot at different velocity caps v_x . **RACER completes every lap and scales its speed as the caps become higher, whereas the naive unicycle finishes laps only at the conservative 1.5 m/s cap and fails entirely (0 laps) at higher speeds.** Clean laps are out of 3 per run; **bold** marks the better value per metric.

velocity cap (m/s)	Method	Clean laps (of 3)	Avg. vel. (m/s)	Max dev. (m)
1.5	RACER	3.0 ± 0.0	1.21 ± 0.01	0.42 ± 0.13
	Naive Unicycle	2.3 ± 1.2	1.49 ± 0.04	0.59 ± 0.12
2.0	RACER	3.0 ± 0.0	1.41 ± 0.02	0.45 ± 0.08
	Naive Unicycle	0.0 ± 0.0	0.00	0.83 ± 0.57
3.2	RACER	3.0 ± 0.0	1.95 ± 0.04	0.35 ± 0.03
	Naive Unicycle	0.0 ± 0.0	0.00	0.80 ± 0.22

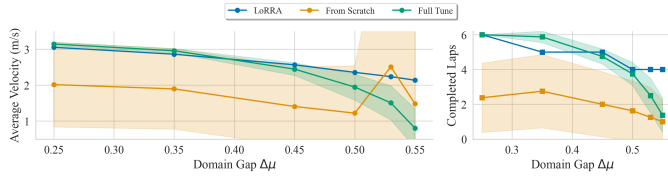


Fig. 7: Performance (Vertical Axis) of RACER using residual dynamics trained with LoRRA (blue), From Scratch (Yellow), Full-Tune (Green) under various domain gaps in terms of friction coefficient (horizontal Axis). **The larger the domain gaps, the greater the advantages hold by LoRRA.** Shaded Area: variance across seeds. For the definition of each training recipe, see Sec. IV-B

naive unicycle fails to complete a single lap, whereas RACER completes all laps, maintains the lowest raceline deviation, and keeps raising its average velocity. This confirms Insight IV.1 on hardware.

B. Study of LoRRA

This section further discusses the training recipe for g_ϕ . We define three strategies for training the residual dynamics:

- (1) **LoRRA**, pre-trained on $\mathcal{D}_s$ and fine-tuned under a low-rank constraint;
- (2) **Full-tune**, pre-trained on $\mathcal{D}_s$ and fine-tuned without rank constraints;
- (3) **From-scratch**, trained only on $\mathcal{D}_t$ with no pretraining. This reflects the adaptation methodology of [26] which trains a fresh residual dynamics with target-domain data.

For this analysis, the residual dynamics are embedded into the RACER architecture and all previous elements of the control system are the same besides the residual's training method. The comparison between LoRRA and Full-tune validates the role of low-rank constraint; From-scratch validates the significance of dual-phase training. We present two insights.

Insight IV.2

Residual dynamics trained with both source-domain and target-domain data gives racing performance significantly better than the residual dynamics trained with only target-domain data.

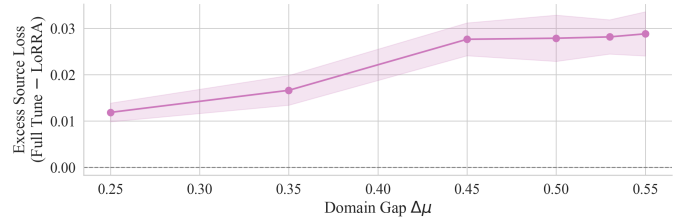


Fig. 8: **The advantage of LoRRA at preserving source-domain posterior increases with domain gap magnitude.** The horizontal axis shows the domain gap $\Delta\mu$. The vertical axis shows the difference in source-domain loss after adapting residual dynamics to $\mathcal{D}_t$, comparing Full-Tune against LoRRA: $\mathcal{L}(\phi, \mathcal{D}_s)_{\text{Full-Tune}} - \mathcal{L}(\phi, \mathcal{D}_s)_{\text{LoRRA}}$.

Insight IV.3

Low-rank constraint at target-domain adaptation gives higher success rates and higher performance than full-tune and from-scratch when domain gap for ground friction or joint gain is large.

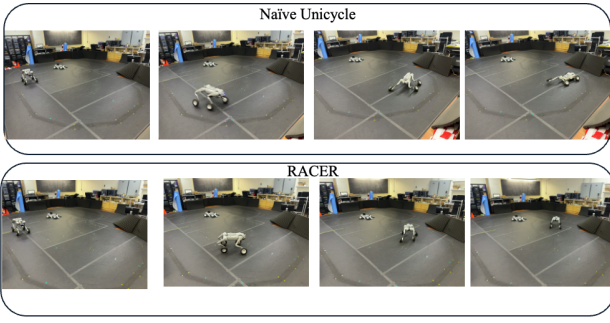
1) *Simulation Results:* For the simulation experiment, we define the target domain as a simulation with dynamics that differ from the source domain. We introduce a controlled gap by changing the tire-ground friction μ_t relative to the source $\mu_s = 0.85$, and measure the gap as $|\mu_s - \mu_t|$. We sweep $\mu_t \in \{0.30, 0.35, 0.40, 0.45, 0.50, 0.55, 0.60, 0.65\}$, running eight independent trials per method at each μ_t , each with its own target-domain data collection and training.

Results are reported in 7. We present two observations:

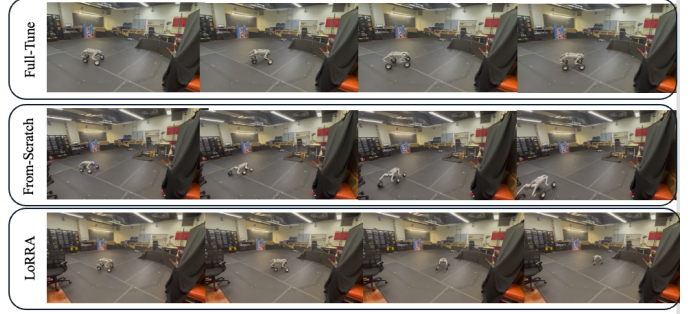
- Compared to Full-tune and LoRRA, From-Scratch demonstrates significantly worse average performance metrics with much higher cross-seed variance. This is consistent with Insight IV.2.
- Compared to Full-tune, LoRRA's advantage grows with the domain gap. Under large gaps, LoRRA reaches higher average velocity and presents more stable lap completion than Full-Tune. This is consistent with Insight IV.3.

The key difference between the three training methods is their ability to preserve the posterior of source-domain pre-training. Fig. 3 validates this. From-scratch cannot equip the residual dynamics with any knowledge about source-domain by its definition so it shows the highest source-domain loss. Compared to Full-tune, LoRRA has lowest source-domain loss after target-domain adaptation because of low-rank constraint and thus LoRRA is the most capable of preserving source-domain knowledge.

In Fig. 8, we further compare LoRRA and Full-tune's ability of preserving source-domain posterior at different domain gaps. It shows that the advantage of LoRRA over full-tune positively correlates with the magnitude of domain gap, which is consistent with the pattern we observe in Fig. 7 and Insight IV.3. Similar observations have also been found in the context of LLM. [33] finds that fine-tuning LLM with low-rank constraint substantially underperforms Full-Tune on the target in high-rank regimes (code) while nearly matching it in lower-



(a) Hardware experiment with velocity cap 3.2 m/s in Sec. IV-A2. Naïve Unicycle (first row) fails at the first corner, while RACER (second row) passes the corner smoothly.



(b) Hardware experiment with joint difference $\Delta K_p = 20$ in Sec. IV-B2. Full-Tune (first row) shakes around the first corner due to over-fitted residual dynamics; From-Scratch (second row) fails before entering the first corner; LoRRA passes the first corner smoothly.

Fig. 9: Hardware ablation experiments for RACER and LoRRA.

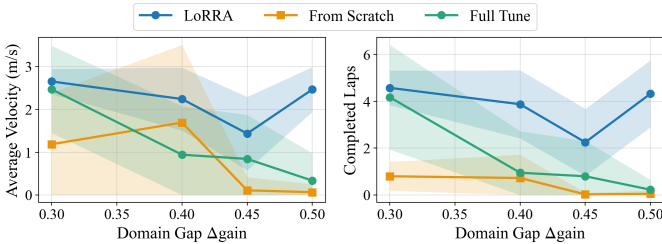


Fig. 10: LoRRA gives performance consistently greater than or equal to From Scratch and Full Tune under different joint-gain domain gaps. This figure is the counterpart of Fig. 7 with domain gaps redefined as joint gain difference instead of friction difference.

gap regimes (math).

To further generalize our results, we repeat the experiment in Fig. 7 by redefining the domain gap as the difference between the joint gains of two domains K_p . LoRRA’s advantage (and the relationship between such advantage and domain gap) observed earlier holds in Fig. 10, but in a much noisier way. The main reason is that domain gap in terms of joint gain is more likely to trigger immediate failure of the policy (falling down to the ground) while domain gap in terms of ground-friction results in smoother dynamics difference.

In conclusion, when the size of target-domain data is essentially small, leveraging source-domain posterior gives us stronger racing performance compared with exclusively using target-domain data (Insight IV.2). When the domain-gap is large, low-rank constraint can help better preserve source-domain posterior and gives better racing performance compared with Full-tune (Insight IV.3).

2) *Hardware Results:* On hardware, we induce a controlled domain gap by varying the low-level joint tracking gain on our hardware, using the same joint-gain gap definition as the simulation study in Fig. 10. The residual is pre-trained with simulation data collected at the nominal gain $K_{p,s} = 70$, and we quantify the sim-to-real gap as $\Delta K_p = |K_{p,s} - K_{p,t}|$ between the simulation joint gain $K_{p,s}$ and hardware joint gain $K_{p,t}$. We test each recipe with $\Delta K_p \in \{0, 30, 50\}$. A larger

TABLE II: **Hardware LoRRA ablation** (counterpart of Fig. 10). Racing metrics (mean $\pm$ std over three independently trained seeds) for the three residual-training recipes under an increasing joint-gain domain gap $\Delta K_p = |K_{p,s} - K_{p,t}|$ (larger ΔK_p = larger gap; the residual is pre-trained at the nominal gain $K_{p,s} = 70$). **LoRRA completes laps robustly across all gaps, whereas Full-Tune degrades as the gap grows and fails entirely at $\Delta K_p = 50$, and From-Scratch fails at nearly every gap.** Velocity is averaged over all seeds (a failed run contributes 0 velocity); **bold** marks the best value per metric.

ΔK_p (gap)	Method	Clean laps (max is 3)	Avg. vel. (m/s)
0	LoRRA	3.00 $\pm$ 0.00	1.94 $\pm$ 0.07
	Full-Tune	2.67 $\pm$ 0.47	2.06 $\pm$ 0.05
	From-Scratch	0.33 $\pm$ 0.47	0.59 $\pm$ 0.83
30	LoRRA	2.33 $\pm$ 0.94	1.61 $\pm$ 0.08
	Full-Tune	1.00 $\pm$ 0.82	0.60 $\pm$ 0.44
	From-Scratch	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00
50	LoRRA	2.33 $\pm$ 0.94	1.29 $\pm$ 0.10
	Full-Tune	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00
	From-Scratch	1.00 $\pm$ 1.41	0.80 $\pm$ 1.14

ΔK_p means larger sim-to-real gap. For each ΔK_p , we collect a hardware trajectory and run three independently-seeded train per recipe using the same hardware dataset. We report the across-seed mean and variance per recipe and per ΔK_p in Table II.

The results reproduce both insights on hardware. From-Scratch, which uses only target-domain data, fails to complete laps at almost every gap and shows the highest across-seed variance, confirming the value of dual-phase training (Insight IV.2). Among the two dual-phase recipes, LoRRA and Full-Tune are comparable at zero gap ($\Delta K_p = 0$), with Full-Tune even slightly faster. As the gap grows, however, Full-Tune degrades sharply while LoRRA keeps completing laps at every gap. This is consistent with Insight IV.3.

V. CONCLUSION

We presented RACER, a hierarchical control framework for autonomous quadruped racing. We exhaustively validated two main ingredients of our pipeline: residual dynamics and

LoRRA. We identify two potential questions for future research. (A) Can hardware adaptation of the residual dynamics happen on-fly safely? (B) How to adapt residual dynamics in wild environments?

ACKNOWLEDGMENT

Acknowledgment withheld for double-blind review.

REFERENCES

- [1] M. Bjelonic, P. K. Sankar, C. D. Bellicoso, H. Vallery, and M. Hutter, “Rolling in the deep—hybrid locomotion for wheeled-legged robots using online trajectory optimization,” *IEEE Robotics and Automation Letters*, vol. 5, no. 2, pp. 3626–3633, 2020.
- [2] J. Lee, M. Bjelonic, A. Reske, L. Wellhausen, T. Miki, and M. Hutter, “Learning robust autonomous navigation and locomotion for wheeled-legged robots,” *Science Robotics*, vol. 9, no. 89, p. eadi9641, 2024.
- [3] S. Katayama, M. Murooka, and Y. Tazaki, “Model predictive control of legged and humanoid robots: models and algorithms,” *Advanced Robotics*, vol. 37, no. 5, pp. 298–315, 2023.
- [4] M. Bjelonic, R. Grandia, O. Harley, C. Galliard, S. Zimmermann, and M. Hutter, “Whole-body mpc and online gait sequence generation for wheeled-legged robots,” in *2021 IEEE/RSJ international conference on intelligent robots and systems (IROS)*. IEEE, 2021, pp. 8388–8395.
- [5] U. Rosolia and F. Borrelli, “Learning how to autonomously race a car: A predictive control approach,” *IEEE Transactions on Control Systems Technology*, vol. 28, no. 6, pp. 2713–2719, 2019.
- [6] R. Kirk, A. Zhang, E. Grefenstette, and T. Rocktäschel, “A survey of zero-shot generalisation in deep reinforcement learning,” *Journal of Artificial Intelligence Research*, vol. 76, p. 201–264, Jan. 2023. [Online]. Available: <http://dx.doi.org/10.1613/jair.1.14174>
- [7] J. Hwangbo, J. Lee, A. Dosovitskiy, D. Bellicoso, V. Tsounis, V. Koltun, and M. Hutter, “Learning agile and dynamic motor skills for legged robots,” *Science robotics*, vol. 4, no. 26, p. eaau5872, 2019.
- [8] N. Rudin, D. Hoeller, P. Reist, and M. Hutter, “Learning to walk in minutes using massively parallel deep reinforcement learning,” in *Conference on robot learning*. PMLR, 2022, pp. 91–100.
- [9] G. Williams, N. Wagener, B. Goldfain, P. Drews, J. M. Rehg, B. Boots, and E. A. Theodorou, “Information theoretic mpc for model-based reinforcement learning,” in *2017 IEEE international conference on robotics and automation (ICRA)*. IEEE, 2017, pp. 1714–1721.
- [10] H. Xue, C. Pan, Z. Yi, G. Qu, and G. Shi, “Full-order sampling-based mpc for torque-level locomotion control via diffusion-style annealing,” in *2025 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2025, pp. 4974–4981.
- [11] P. Roth, J. Frey, C. Cadena, and M. Hutter, “Learned perceptive forward dynamics model for safe and platform-aware robotic navigation,” 2025.
- [12] Y. Kim, C. Kim, and J. Hwangbo, “Learning Forward Dynamics Model and Informed Trajectory Sampler for Safe Quadruped Navigation,” in *Proceedings of Robotics: Science and Systems*, New York City, NY, USA, June 2022.
- [13] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel, “Domain randomization for transferring deep neural networks from simulation to the real world,” in *2017 IEEE/RSJ international conference on intelligent robots and systems (IROS)*. IEEE, 2017, pp. 23–30.
- [14] A. Kumar, Z. Fu, D. Pathak, and J. Malik, “RMA: Rapid Motor Adaptation for Legged Robots,” in *Proceedings of Robotics: Science and Systems*, Virtual, July 2021.
- [15] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, “LoRA: Low-rank adaptation of large language models,” in *International Conference on Learning Representations*, 2022.
- [16] E. Daneshmand, S. Omar, G. Berseth, M. Khadiv, and H.-C. Lin, “Slowrl: Safe low-rank adaptation reinforcement learning for locomotion,” *arXiv preprint arXiv:2603.17092*, 2026.
- [17] M. Eisman, B. Lam, S. Sonnino, and F. Borrelli, “Racing a wheeled quadruped: Active load transfer mitigation via model predictive control,” *arXiv preprint arXiv:2606.26313*, 2026, accepted to the 17th International Symposium on Advanced Vehicle Control (AVEC 2026).
- [18] T. Kato, K. Oishi, and S. Ito, “Model predictive path integral pid control for learning-based path following,” 2026.
- [19] S. Song, W. Mei, and C. Zhao, “Robust path tracking for vehicles via continuous-time residual learning: An icode-mppi approach,” 2026.
- [20] C. Pezzato, C. Salmi, E. Trevisan, M. Spahn, J. Alonso-Mora, and C. H. Corbato, “Sampling-based model predictive control leveraging parallelizable physics simulations,” *IEEE Robotics and Automation Letters*, vol. 10, no. 3, pp. 2750–2757, 2025.
- [21] E. Heiden, D. Millard, E. Coumans, Y. Sheng, and G. S. Sukhatme, “Neuralsim: Augmenting differentiable simulators with neural networks,” in *2021 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2021, pp. 9474–9481.
- [22] S. Cheng, M. Kim, L. Song, C. Yang, Y. Jin, S. Wang, and N. Hovakimyan, “Diffune: Autotuning through autodifferentiation,” *IEEE Transactions on Robotics*, vol. 40, pp. 4085–4101, 2024.
- [23] L. Krishna, S. Cheng, J. Li, N. Hovakimyan, and Q. Nguyen, “Diffcotune: Differentiable co-tuning for cross-domain robot control,” *IEEE Robotics and Automation Letters*, 2025.
- [24] T. He, J. Gao, W. Xiao, Y. Zhang, Z. Wang, J. Wang, Z. Luo, G. He, N. Sobanbabu, C. Pan, Z. Yi, G. Qu, K. Kitani, J. K. Hodgins, L. Fan, Y. Zhu, C. Liu,

and G. Shi, “ASAP: Aligning Simulation and Real-World Physics for Learning Agile Humanoid Whole-Body Skills,” in *Proceedings of Robotics: Science and Systems*, Los Angeles, CA, USA, June 2025.

- [25] J. Levy, T. Westenbroek, and D. Fridovich-Keil, “Learning to walk from three minutes of real-world data with semi-structured dynamics models,” in *Proceedings of The 8th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, P. Agrawal, O. Kroemer, and W. Burgard, Eds., vol. 270. PMLR, 06–09 Nov 2025, pp. 2061–2079.
- [26] E. Lee, S. A. Moore, and B. Chen, “Sym2real: Symbolic dynamics with residual learning for data-efficient adaptive control,” *arXiv preprint arXiv:2509.15412*, 2025.
- [27] Z. Fan, “robot_lab: RL extension library for robots, based on IsaacLab,” https://github.com/fan-ziqi/robot_lab, 2024.
- [28] X. B. Peng, M. Andrychowicz, W. Zaremba, and P. Abbeel, “Sim-to-real transfer of robotic control with dynamics randomization,” in *2018 IEEE international conference on robotics and automation (ICRA)*. IEEE, 2018, pp. 3803–3810.
- [29] M. Buda, A. Maki, and M. A. Mazurowski, “A systematic study of the class imbalance problem in convolutional neural networks,” *Neural Networks*, vol. 106, p. 249–259, Oct. 2018. [Online]. Available: <http://dx.doi.org/10.1016/j.neunet.2018.07.011>
- [30] J. Byrd and Z. C. Lipton, “What is the effect of importance weighting in deep learning?” 2019. [Online]. Available: <https://arxiv.org/abs/1812.03372>
- [31] T. Yu, S. Kumar, A. Gupta, S. Levine, K. Hausman, and C. Finn, “Gradient surgery for multi-task learning,” 2020. [Online]. Available: <https://arxiv.org/abs/2001.06782>
- [32] B. Liu, X. Liu, X. Jin, P. Stone, and Q. Liu, “Conflict-averse gradient descent for multi-task learning,” 2024. [Online]. Available: <https://arxiv.org/abs/2110.14048>
- [33] D. Biderman, J. Portes, J. J. G. Ortiz, M. Paul, P. Greengard, C. Jennings, D. King, S. Havens, V. Chiley, J. Frankle, C. Blakeney, and J. P. Cunningham, “Lora learns less and forgets less,” 2024. [Online]. Available: <https://arxiv.org/abs/2405.09673>
- [34] R. Shuttleworth, J. Andreas, A. Torralba, and P. Sharma, “Lora vs full fine-tuning: An illusion of equivalence,” 2025. [Online]. Available: <https://arxiv.org/abs/2410.21228>
- [35] X. Wang, T. Chen, Q. Ge, H. Xia, R. Bao, R. Zheng, Q. Zhang, T. Gui, and X. Huang, “Orthogonal subspace learning for language model continual learning,” 2023. [Online]. Available: <https://arxiv.org/abs/2310.14152>

APPENDIX

A. DIAL-MPC Cost Function

The DIAL-MPC planner optimizes trajectories to stay closely to a globally optimum reference trajectory computed offline. When a reference trajectory $(x^*(s), y^*(s), \psi^*(s), v^*(s))$ parameterised by arclength s is available, the contouring term is replaced by an explicit trajectory-tracking error. For each rollout state $s_h^{(k)} = (x_h^{(k)}, y_h^{(k)}, \psi_h^{(k)}, v_{x,h}^{(k)}, \omega_{z,h}^{(k)})$, we project $(x_h^{(k)}, y_h^{(k)})$ to its closest arclength using the same grid projector, query the reference at that arclength, and sum

$$J_{\text{track}}^{(k)} = \lambda_c \sum_{h=0}^H (\Delta x_h^2 + \Delta y_h^2 + \Delta \psi_h^2) + \lambda_v \sum_{h=0}^H \Delta v_{x,h}^2, \quad (3)$$

with $\Delta x_h = x^*(s_h^{(k)}) - x_h^{(k)}$ and analogous definitions for $\Delta y_h, \Delta \psi_h, \Delta v_{x,h}$. The reference trajectory itself is precomputed offline by solving a nonlinear program on a dynamic-bicycle model of the Go2-W in CasADi with velocity cap $v_x \leq c$ (Sec. IV-A1). The maximum deviation metrics in Sec. IV is computed relative to this reference trajectory.

B. Details on LoRRA

Denote the parameter matrix of g_ϕ ’s l th layer as $W_l \in \mathbb{R}^{d \times k}$. The update rule is parameterized as:

$$W_l := W_l^0 + \frac{\alpha}{r} BA \quad (4)$$

where W_l^0 is the pretrained weights from source-domain which is frozen throughout target-domain adaptation. $B \in \mathbb{R}^{d \times r}$ and $A \in \mathbb{R}^{r \times k}$ can be updated throughout the back-propagation passes of target-domain adaptation. $r \leq \min(d, k)$ is the adaptable rank.

Two hyperparameters are of our interest: r and α . r decides the degree of freedom allowed for adaptation, and α decides the magnitudes of the update.

To simplify our hyperparameter tuning, we assume each layer has the same r and α value. Following LLM community’s common practice of hyper-parameter tuning [15, 33], α is fixed to be 0.35 and we sweep through r in the candidates $r \in \{2, 4, 6, 8, 10, 12\}$ and pick the one that can give the best performance for sim-to-sim domain adaptation with ground-friction domain gap $\Delta\mu = 0.55$. We found $r = 6$ to give the best performance. The hyper-parameter pair $r = 6$ and $\alpha = 0.35$ is used throughout for all the sim-to-sim and sim-to-real adaptations in IV. Under this setting, the number parameters being updated are 17.7 percent of the total parameters of g_ϕ .